PA-Project

Basel Kelziye

January 2026

1 Micro architecture

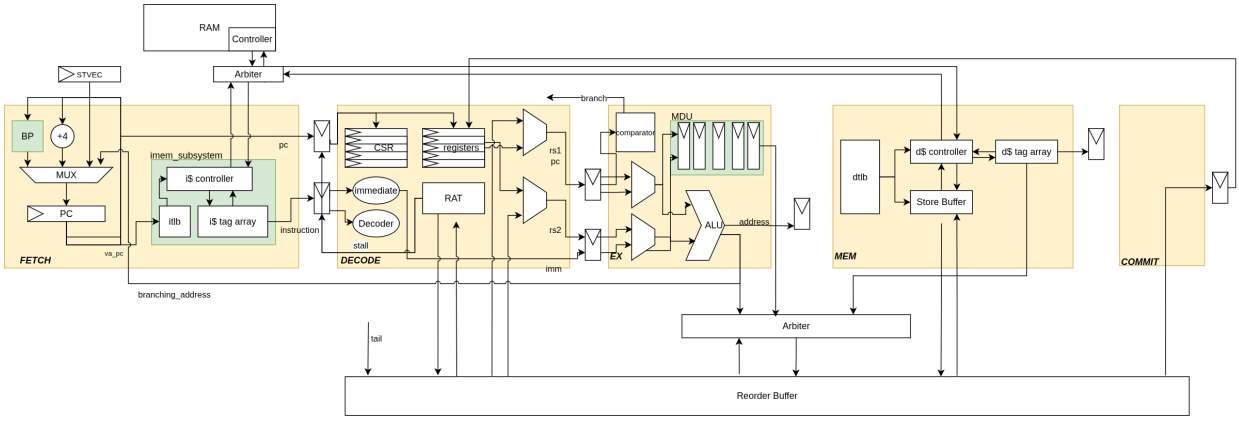


Figure 1: Microarchitecture Diagram

The core has the following Features:

- 2 Bit Branch Predictor
- 32 Lines direct mapped icache and dcache, 16 Bytes per line
- direct mapped itlb and dtlb, Virtual Memory
- Supervisor and User mode, page walk table in software
- Control and Status Registers
- Forwarding from ROB and Store Buffer Only
- Single Trap Handler code, differentiates exceptions with special CSR
- Reorder Buffer

2 Load Instructions

IN decode stage, The load instructions gets allocated in Reorder Buffer and Register Aliases Table, In EX stage the address gets generated, and in Memory stage, the dTLB is checked if we have a valid address translation and concurrently the following is checked: The address checking happens concurrently with the value reading, because i have Virtually Indexed Physically Tagged Caches, If dtlb translation is not valid, exception will be raised and the value read from cache will be considered garbage and will not be issued. If the translation is valid: First the store buffer will be checked, if there is a hit, store buffer forwarding will happen. If data is not available in store buffer, dcache controller will take in and will look if its available in the tag array. If yes then the specific bytes will be read from the line, if not then a request to memory will happen. If the line is marked as dirty (cache line is different than memory) then cache evit will happen. if not, a line fill request will happen. dcache requests are more priotrized than the icache requests, if there is a contention arbiter will gran for dcache. However if there is an outgoing icache transaction, the arbiter will not intervent it and it will wait it untill it finished. Finally the instrction gets committed when its in the head of the ROB.

3 Store Instructions

In Decode stage, The Store instructions gets allocated in the Reorder Buffer, in EX stage the address gets generated, and in Memory stage the store is stored directly in the store buffer **without** checking for valid translation nor checking the tag arrays. If store buffer is full, stall will happen until store buffer drains. If we have a dtlb miss, the exception in the respective store's entry is raised. And when its the oldest instruction it will trap. Consider we have a store buffer forwarding from a faulty store address translation, the load will get a faulty value, but since its younger than the store it will get flushed as well. When the store is the oldest instruction in the ROB, if we have no exception a drain request happen. it will drain oldest store from store buffer to dcache tag array, the eviction,fill happens here. This action stalls the processor as my project supports blocking caches.

The Reorder buffer has a valid/ready handshake with Store buffer to issue a drain request.

Then the Store buffer also issues a store to tag array with dcache controller using ready/valid handshake.

4 Branch/Jump Instructions

The project supports 2 bit branch predictor thats initialized as empty (Not Taken). The Branch Predictor takes the Virtual Addressed program counter as input and outputs virtual addressed program counter. If itlb miss happens, the branch instruction will trap. In EX stage the predictor is updated based on our decision made and the actualy branch result. Here i want to note that the Branch Predictor is trained at EX stage while Branch instruction is committed later in time, depends on rob, this can cause some faulty branches to train the BP (maybe they will never execute because of exception, consecutive branches etc). However since this only effects the predictor's decision, it will not effect our project's correctness but it might lower the predictor's accuracy. Also, i have turned the branch predictor while Supervisor mode is executing because there an overlap between virtual address and physical address can happen.

For example my Trap handler code relies in 0x0000-0x5000. If a branch was in virtual addresss 0x1000 and the state of predictor was taken for example. Then in the trap handler also the Real physicall address was 0x1000 and it was also a branch, the predictor will branch and will switch me from Supervisor code to User code without SRET, and this is a little bit dangerous.

Finally, when a branch happens, its treated as an exception, every instruction behind gets flushed

5 Arithmetic

In Decode stage, the Arithmetic instructions gets allocated in Reorder Buffer and in Register Aliases table. Multiplications are issued in MDU unit, its fully pipelined can accept 5 instructions in flight. Other operations are done in single cycle, then the result is validated in the ROB. There is an arbitration for the ROB to determine if value is written from Memory, MDU or ALU. Explained in ROB Section.

6 Exceptions + ROB

The Reorder Buffer can be conceptually divided in 3 stages, First an instruction gets allocated, this is done in decode stage and in an in-order fashion, an instruction receives an **rob_id** and keeps it untill its commited. Second stage is when the result gets completed, for example when the ALU operations ends usually it used to update the register file directly but now its write its operation to ROB and mark the entry as valid which means it can be committed when its the oldest instructions in the ROB. In this validating step the exceptions are flagged if there is any, and and scause register is also set to help the trap handler know the reason why its trapped. Hence we have a single trap handler that handles all the exceptions. Finally when an instruction is marked as valid and its done, we commit it if it doesn't have any exceptions, if it does we trap the hander. The PC gets set to the STVEC value that is set already, and it drived the SCAUSE from CSR Unit. As mentioned in the Store instructions, the commit is different, the commit is actually a drain request from the Store Buffer, if we don't have exceptions.

7 Virtual Memory

The program relies in 0x0 Virtual address, Trap handler in 0x0 Physical Adress. I support 2 level of page tables. SATP register points to the L2 (outer) page level. Each entry of this table maps to a L1 Page tables. The page table walking is done in trap handler. when and itlb/dtlb miss happens the trap handler jumps in. A trap handler can be cause because:

1. L2 Page is not mapped
2. L1 Page is not mapped
3. Page mapping is available but its not available in TLB

L2 Page mapping: Starting address being from 0x5000, each page entry is reserved for the next 0x1000, for example the address in 0x5000 gets translated to 0x6000, and the address for 0x5004 is translated to 0x7000. Because each page table entry is 12 bits, we need 0x1000 for each entry.

L1 Mapping: Here once we are in the respective leaf for each page table entry, **PA = VA + 0x41000** mapping is done. for example program binary is located at VA = 0x0, but in physicall memory it gets translated to 0x41000.

Also 2 new priviledged instructions are added, itlb write and dtlb write. They take a virtual address, and the mapping value and writes them to itlb and dtlb respectively. This is done only when the processor is in supervisor mode. Size concerns: Page tables are reserving addresses from 0x5000-0x405FFF which is equal to 4MB!

```
{Software Page Table Walker}
cause ← read_csr(scause)
if cause ≠ 12 and cause ≠ 13 and cause ≠ 15 then
    goto exit
end if
vAddr ← read_csr(stval)
root_base ← read_csr(satp).PPN ≪ 12
{— Level 1 Walk —}
vpn1 ← vAddr[31 : 22]
pte1_pa ← root_base + (vpn1 × 4)
pte1 ← memory[pte1_pa]
if (pte1 & 1) = 0 then
    {Level 1 Allocation}
    l2_pa ← root_base + ((vpn1 + 1) × 4096)
    pte1 ← (l2_pa ≫ 2) | 1 {Set Valid bit}
    memory[pte1_pa] ← pte1
end if
{— Level 2 Walk —}
l2_base ← (pte1 ≫ 10) ≪ 12
vpn0 ← vAddr[21 : 12]
pte2_pa ← l2_base + (vpn0 × 4)
```

21: $pte2 \leftarrow memory[pte2_pa]$
22: **if** $(pte2\ \&\ 1) = 0$ **then**
{Level 2 Allocation (Data Mapping)}
23: $physAddr \leftarrow vAddr + 0x410000$
24: $pte2 \leftarrow (physAddr \gg 2) \mid 0xCF$ {Set V, R, W, X, A, D}
25: $memory[pte2_pa] \leftarrow pte2$
26: **end if**
27: {— Final TLB Update —}
28: **if** $cause = 12$ **then**
29: **ITLB_WRITE**$(vAddr, pte2)$
30: **else**
31: **DTLB_WRITE**$(vAddr, pte2)$
32: **end if**

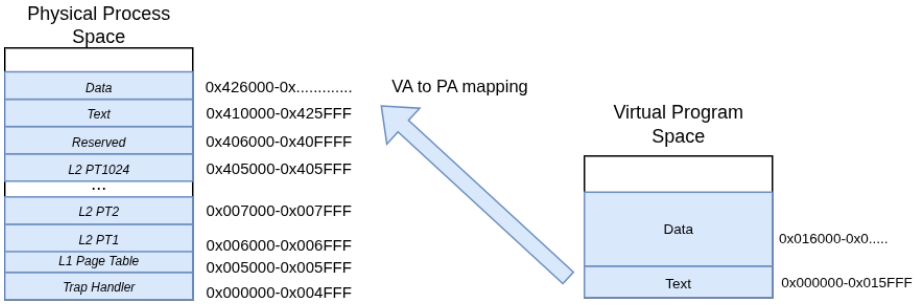


Figure 2: Process and Program Address Spaces

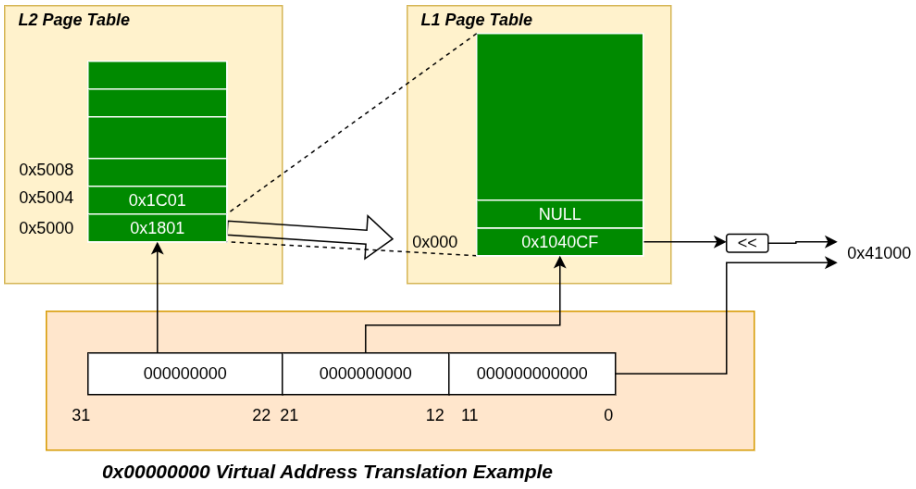
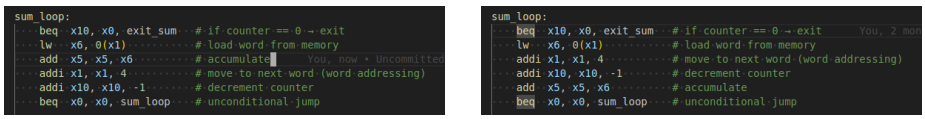


Figure 3: 2 Level Page Tables Walk Example

8 Performance Analysis

at the end of each test i injected a **jal x0, 0** instruction to cause a endless loop, when ever the first **jal x0, 0** gets committed i know my test is done.

To increase performance , i have modified the program code to reduce the latencies, in Figure 4 Can be seen that for each loop 2 Cycles are saved. For



(a) Original Sum Performance (b) Enhanced Sum Performance

Figure 4: Comparison of Performance Enhancements for the Sum Algorithm

the matrix multiplication, the code can be modified to reduce the cache lines contention but due to time conerncs this left as a TODO. The Performance analysis can be seen in Table 1.

Table 1: Core Performance Metrics Comparison

Test Name	**Total Cycles**	**Instructions**	**CPI**	**IPC**
buffer_sum_test	2,627	764	3.438	0.291
mem_copy_test	7,520	1,469	5.119	0.195
matrix_mul	59,984,805	16,876,689	3.554	0.281